

Titanium Sponsors



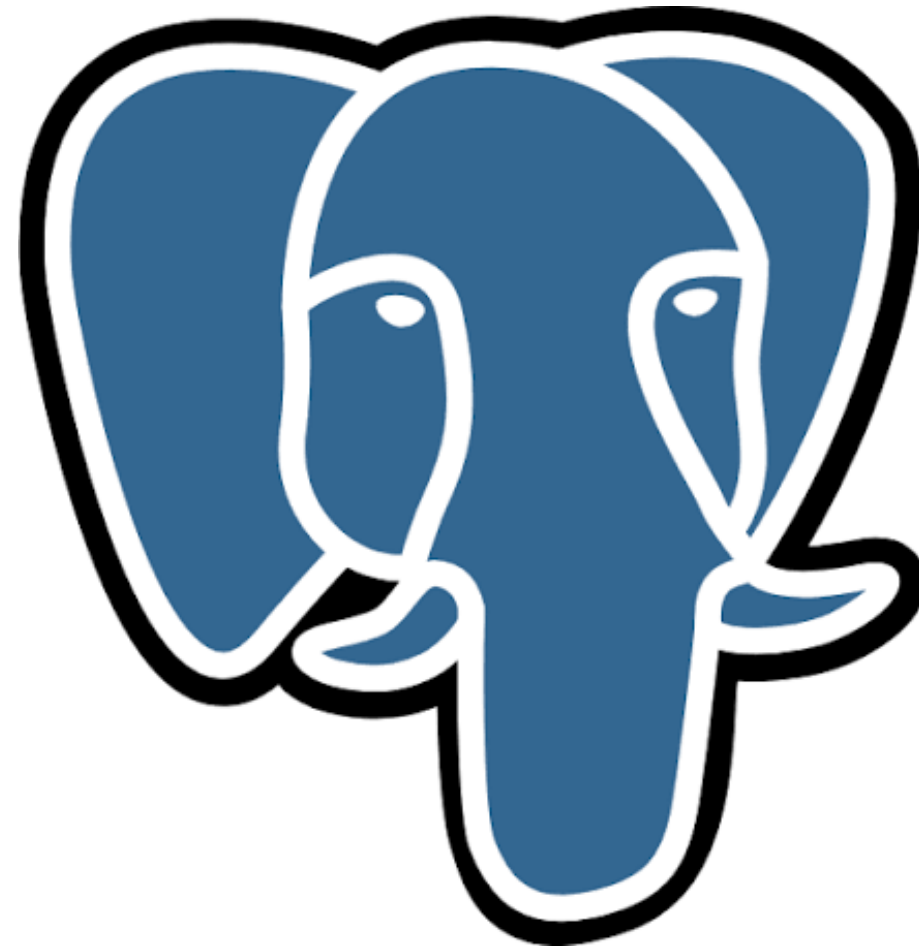
Other Awesome Organizations

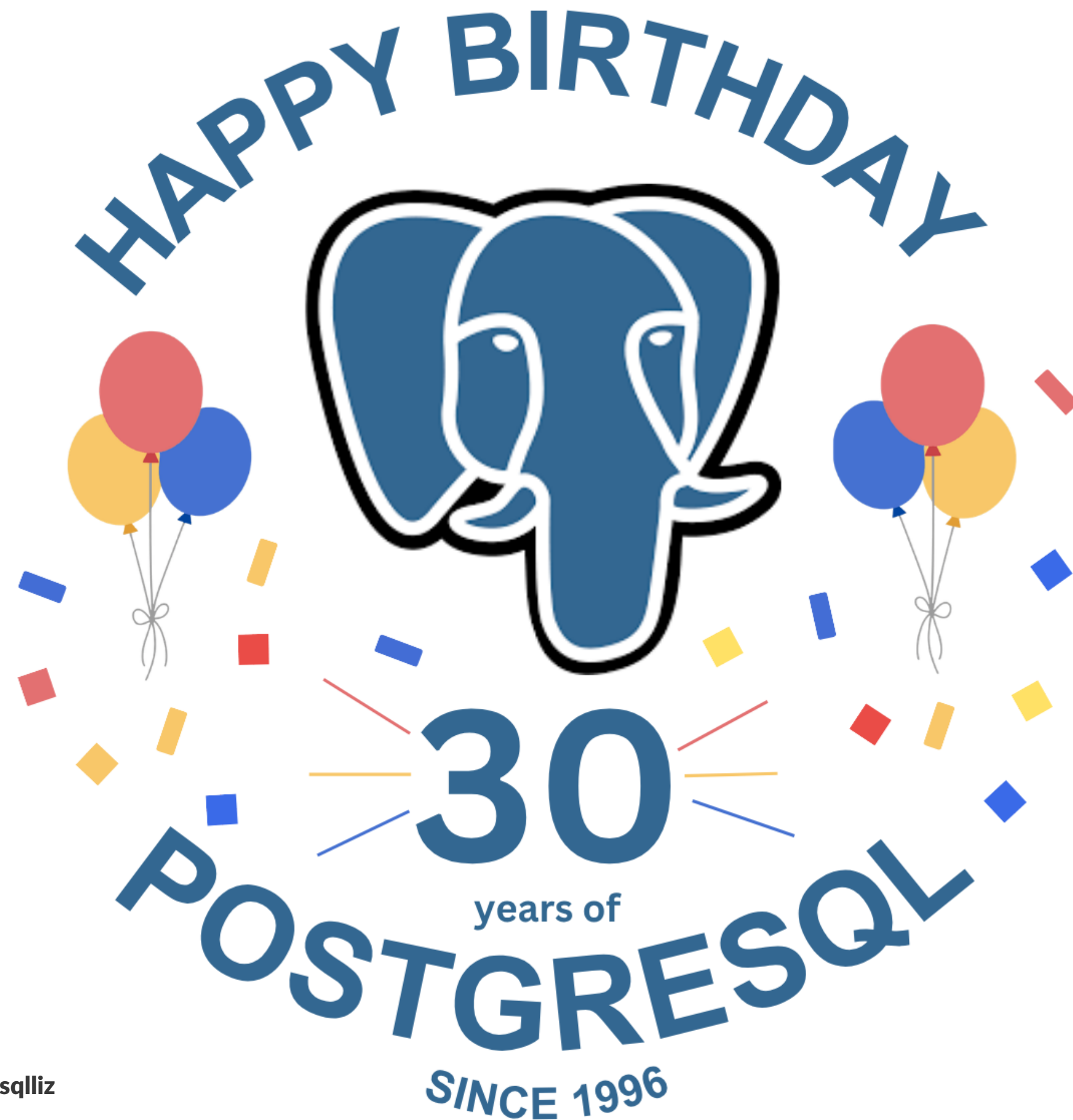
Just Use Postgres | KCDC 2026 | @salliz



DevIT:>_
DevITJobs.com

Just Use Postgres: A Strategy for Modern Apps





About Me

Elizabeth Christensen

- Developer Advocate @ Snowflake
- PostgreSQL contributor & pgUS Board Member
- KC Postgres + Postgres Meetup for All



KC OPEN SOURCE NIGHT

**SEPT
10**

7:00 - 9:00PM

**LARKS ENTERTAINMENT
@ WEST BOTTOMS**

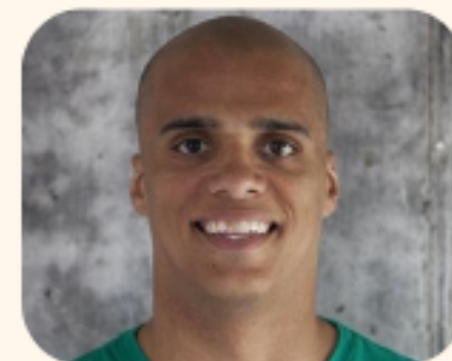
STACEY HAYSLER

Postgres



DASHAUN CARTER

Spring



FRANK WILES

Django



DEREK MOORE

Hermes Agent



Data tools and possible consolidation

Need	"Proper" Tool
Job queues	Redis, Sidekiq, Celery
Events/Pub-Sub	Kafka, RabbitMQ
Caching	Redis, Memcached
Document store	MongoDB
Spatial	Esri / ArcGIS
Graph DB	Neo4j
Vector search	Pinecone
Full-text search	Elasticsearch
OLAP	ClickHouse, DuckDB

The Question

Can Postgres actually replace these?

Spoiler: Sometimes yes, sometimes "it depends"

Our Pattern Today

1. **The problem** — what do you need?
2. **The usual tool** — what people reach for
3. **The Postgres way** — how PG does it
4. **Code examples** — using Bluebox data
5. **Tradeoffs** — when to stay, when to leave

Talk slides and markdown

- `github.com/sfc-gh-echristensen/bluebox`



Our Demo Data: Bluebox

- Movie rental database (think: Redbox but blue 🐘)
- Real TMDB movie data, PostGIS geography
- Film metadata with JSONB, tsvector, arrays
- [Bluebox Postgres sample data](#)
- More at [Postgres Full Day Training](#)

Bluebox Schema

<code>bluebox.film</code>	-- title, overview, tsvector, JSONB
<code>bluebox.customer</code>	-- full_name, email, geog (PostGIS)
<code>bluebox.store</code>	-- street_name, geog (PostGIS)
<code>bluebox.rental</code>	-- rental_period (tstzrange)
<code>bluebox.payment</code>	-- amount, payment_date
<code>bluebox.inventory</code>	-- film_id, store_id
<code>bluebox.person</code>	-- actors/crew
<code>bluebox.film_cast</code>	-- film ↔ person relationships

Extensibility

Why Postgres can do all this

The Original Vision

"The Postgres data model ... provides the user with a powerful and flexible tool ... the system is **extensible** by the user in every way."

— *Stonebraker & Rowe, 1986*

Extensions vs. Forks

Approach	Examples	Tradeoff
Core	tsvector, JSONB, ranges	Always available
Contrib	<i>pgtrgm</i> , <i>hstore</i> , <i>pgstat_statements</i>	Ships with PG
Extensions	PostGIS, pgvector, TimescaleDB	Install separately
Forks	Yugabyte, Citus	Powerful, but moves away from core PG

Installing an Extension

-- It's this easy

```
CREATE EXTENSION IF NOT EXISTS postgis;  
CREATE EXTENSION IF NOT EXISTS pg_trgm;  
CREATE EXTENSION IF NOT EXISTS vector;
```

-- See what you have

```
SELECT * FROM pg_available_extensions  
WHERE installed_version IS NOT NULL;
```

Popular extensions

- **PostGIS** — spatial
- **pgvector** — embeddings & similarity search
- **Citus** — sharding
- **TimescaleDB** — time series hypertables
- **Apache AGE** — graph queries (Cypher)
- **pg_cron** — scheduled jobs

Job Queues

SKIP LOCKED & Advisory Locks

The Problem: Overdue Rentals

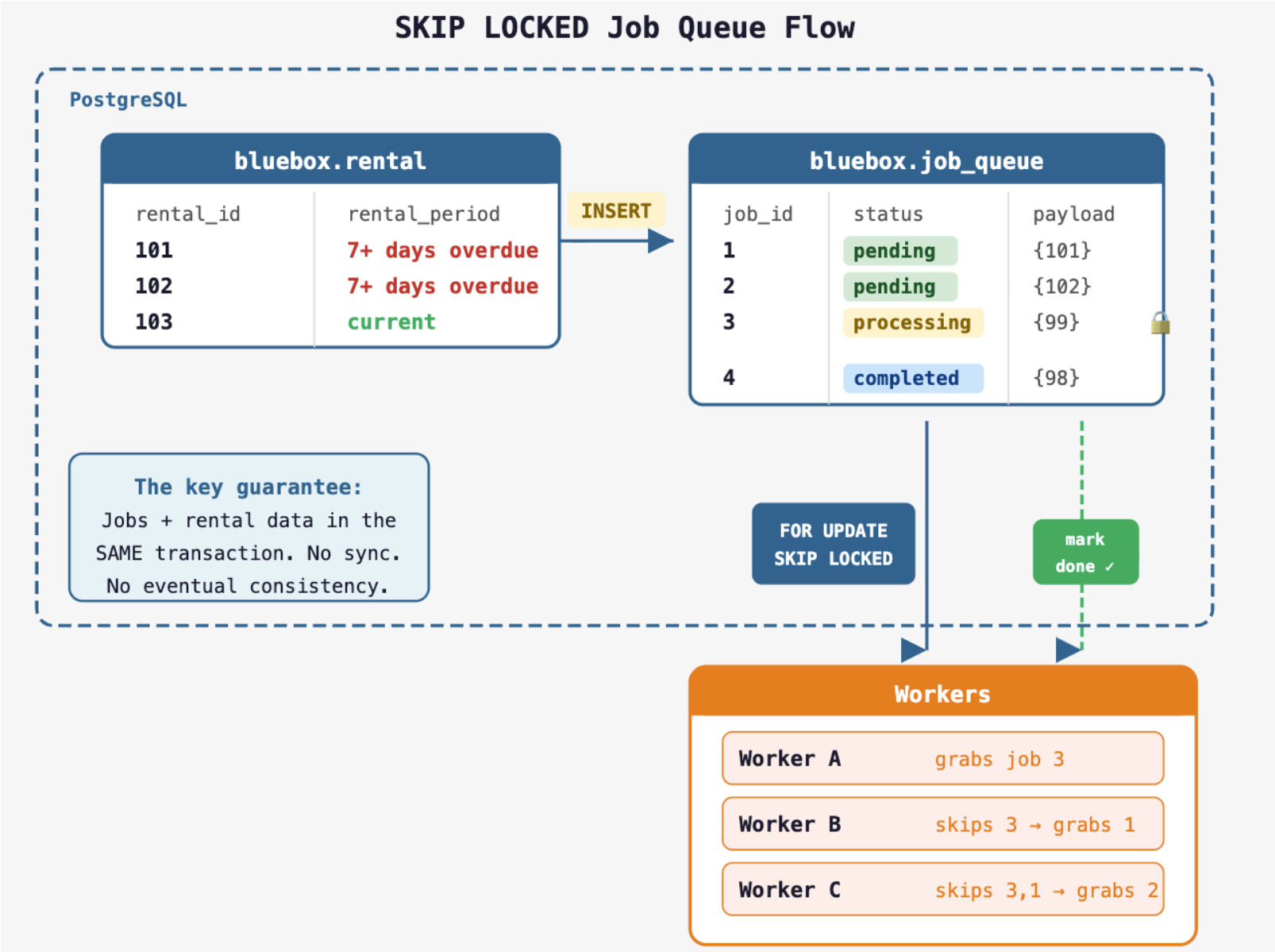
- DVDs are 7+ days overdue — need to process return notices
- Multiple workers handling notices, can't double-send
- If a notice fails, retry it automatically
- Jobs and rental data live in the same database

Job Queues: The Usual Suspects

- **Redis + Sidekiq** (Ruby)
- **Redis + BullMQ** (Node.js)
- **Celery + RabbitMQ/Redis** (Python)
- **Amazon SQS**

All require a separate service to operate.

Postgres 'FOR UPDATE' 'SKIP LOCKED'



Create a Job Queue Table

```
CREATE TABLE bluebox.job_queue (  
    job_id BIGSERIAL PRIMARY KEY,  
    job_type TEXT NOT NULL,  
    payload JSONB,  
    status TEXT DEFAULT 'pending',  
    created_at TIMESTAMPTZ DEFAULT now(),  
    attempted_at TIMESTAMPTZ,  
    attempts INT DEFAULT 0  
);
```

Move things to the process table

```
-- Queue up overdue rental returns
INSERT INTO bluebox.job_queue (job_type, payload)
SELECT 'process_return',
       jsonb_build_object(
         'rental_id', rental_id,
         'customer_id', customer_id,
         'days_overdue', EXTRACT(DAY FROM now() - upper(rental_period))
       )
FROM bluebox.rental
WHERE upper(rental_period) < now() - interval '7 days'
      AND rental_id NOT IN (
        SELECT (payload->>'rental_id')::int FROM bluebox.job_queue
        WHERE job_type = 'process_return'
      );
```

Claim a Job (SKIP LOCKED)

```
-- Worker grabs ONE job, skipping locked rows
UPDATE bluebox.job_queue
SET status = 'processing',
    attempted_at = now(),
    attempts = attempts + 1
WHERE job_id = (
    SELECT job_id FROM bluebox.job_queue
    WHERE status = 'pending'
    ORDER BY created_at
    FOR UPDATE SKIP LOCKED
    LIMIT 1
)
RETURNING *;
```

Step 4: To the work and upate

```
-- Success  
UPDATE bluebox.job_queue SET status = 'completed' WHERE job_id = 123;
```

Job queue libraries

- [GoodJob](#) (Ruby/Rails) — PG-backed ActiveJob
- [Solid Queue](#) (Rails 8 default) — SKIP LOCKED
- [PgQueuer](#) (Python) — LISTEN/NOTIFY + SKIP LOCKED
- [Graphile Worker](#) (Node.js) — advisory locks
- [Que](#) (Ruby) — advisory locks, 10K jobs/sec

Job Queue Benchmarks

- [Que](#) — 10,000 jobs/sec with PostgreSQL advisory locks
- [Graphile Worker](#) — sub-3ms dispatch latency, 100-200 jobs/sec on typical hardware

Job Queues: Tradeoffs

Postgres

~50K jobs/min

Transactional consistency

One less service

Same backups, same monitoring

Your data and jobs are atomic

Use Redis/SQS when...

~100K jobs/min

Ultra-low latency needed

Advanced routing/priorities

Multi-region distribution

Job payload > few KB

Events & Pub/Sub

LISTEN / NOTIFY

The Problem: Real-Time Rental Alerts

- New rental comes in — update the store dashboard instantly
- Notify the inventory system when stock changes
- Trigger a receipt email without polling

Events: The Usual Suspects

- **Apache Kafka** — event streaming at scale
- **Redis Pub/Sub** — fast, ephemeral
- **RabbitMQ** — message routing
- **AWS SNS/SQS** — managed cloud

Postgres Has: LISTEN / NOTIFY

-- Session 1: Subscribe to a channel

```
LISTEN new_rental;
```

-- Session 2: Publish an event

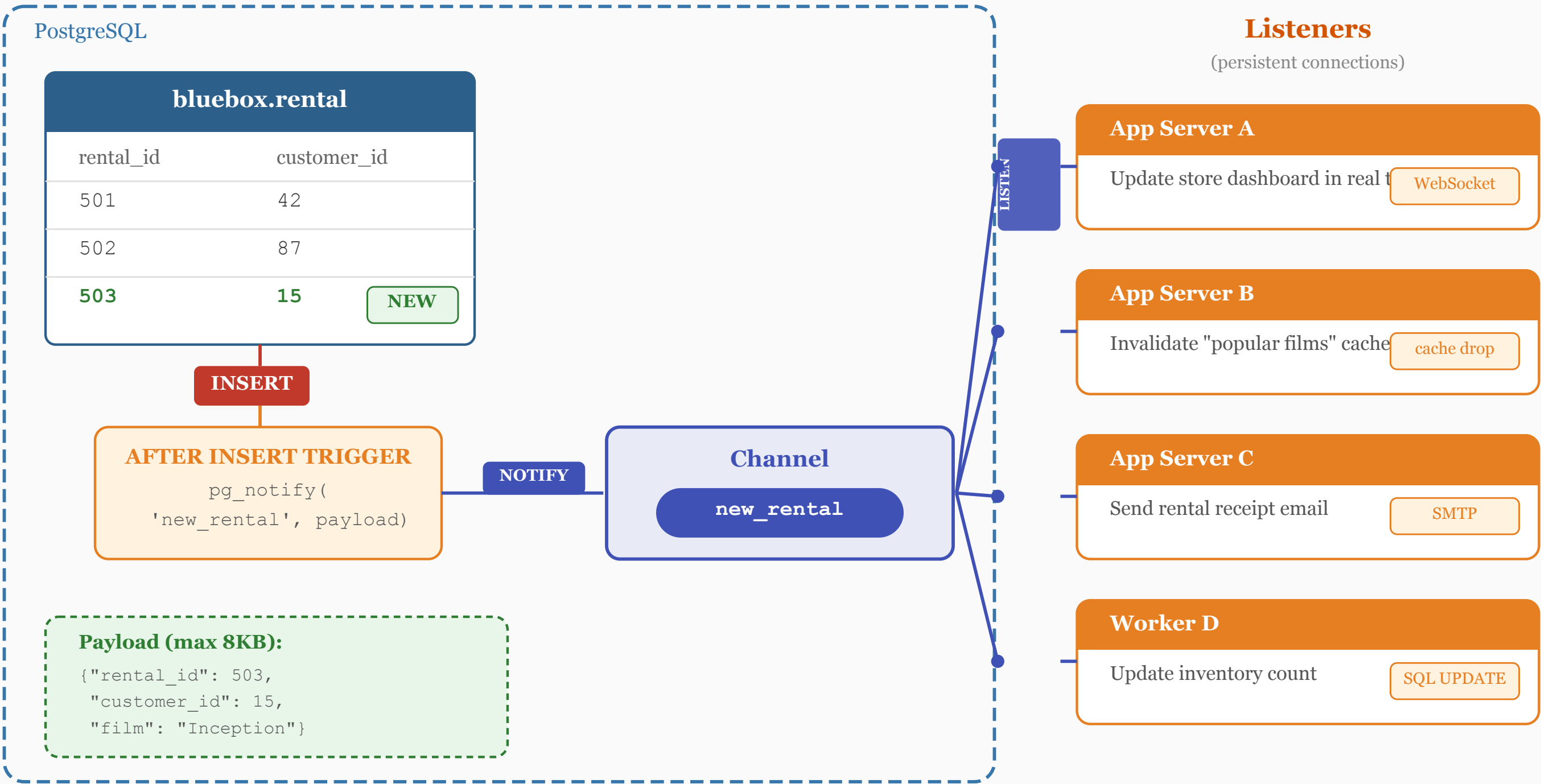
```
NOTIFY new_rental, '{"customer": "Jane", "film": "Inception"}';
```

-- Session 1 receives:

-- Asynchronous notification "new_rental" with payload

```
-- '{"customer": "Jane", "film": "Inception"}'
```

LISTEN / NOTIFY Pub/Sub Flow



Auto-Notify on INSERT via Trigger

```
CREATE TRIGGER rental_notify
  AFTER INSERT ON bluebox.rental
  FOR EACH ROW EXECUTE FUNCTION notify_new_rental();
```

```
CREATE OR REPLACE FUNCTION notify_new_rental()
  RETURNS TRIGGER AS $$
  BEGIN
    PERFORM pg_notify('new_rental',
      json_build_object(
        'rental_id', NEW.rental_id,
        'customer_id', NEW.customer_id,
        'inventory_id', NEW.inventory_id,
        'rented_at', lower(NEW.rental_period)
      )::text
    );
    RETURN NEW;
  END;
$$ LANGUAGE plpgsql;
```

Listening in Application Code (Python)

```
import psycopg2
import select

conn = psycopg2.connect("dbname=bluebox")
conn.autocommit = True
cur = conn.cursor()
cur.execute("LISTEN new_rental;")

while True:
    select.select([conn], [], [], 5) # 5s timeout
    conn.poll()
    while conn.notifies:
        notify = conn.notifies.pop(0)
        print(f"New rental: {notify.payload}")
        # Send webhook, update cache, etc.
```

Real-World: Forem (dev.to)

"We considered building pub/sub on Redis, on Pusher or any other third party system but for the scope of this feature we decided against it."

— *Forem GitHub Issue #3715*

They chose PostgreSQL LISTEN/NOTIFY for webhook delivery.

github.com/forem/forem/issues/3715

Limits of Postgres Listen/Notify

- **~1,000 notifications/sec** sustained max
- Payload size limited to **8,000 bytes**
- No **pattern subscriptions** (Redis has PSUBSCRIBE)
- Listeners need **persistent connections**
- Messages are **fire-and-forget** (not durable)
- Single instance only (no replication)

Events: Tradeoffs

Postgres LISTEN/NOTIFY

Signaling: cache invalidation

Job wakeup triggers

Live UI refresh

Zero extra infrastructure

Transactional delivery

Use Kafka/Redis when...

High throughput (>1K msg/sec)

Message durability required

Multi-consumer replay

Cross-service event bus

Pattern subscriptions needed

Going Further: pgsql-http

```
-- Send actual HTTP webhooks from inside Postgres!  
CREATE EXTENSION http;  
  
-- POST a webhook from a trigger  
CREATE OR REPLACE FUNCTION webhook_on_rental()  
RETURNS TRIGGER AS $$  
BEGIN  
    PERFORM http_post(  
        'https://api.example.com/webhooks/rental',  
        json_build_object(  
            'event', 'new_rental',  
            'rental_id', NEW.rental_id,  
            'customer_id', NEW.customer_id  
        )::text,  
        'application/json'  
    );  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;
```

pg_eventserv: WebSockets from Postgres

- Bridges LISTEN/NOTIFY to **WebSockets** for web clients
- Real-time maps, dashboards, auction systems, fleet tracking
- Zero application middleware — events go DB → browser
- Crunchy Data open-source project

For Durable Events: CDC

If you need guaranteed delivery, look at:

- **Debezium** — reads WAL, writes to Kafka
- **Postgres logical replication** — built-in replication
- **Triggers + outbox table** — transactional outbox pattern

Caching & Key-Value UNLOGGED Tables

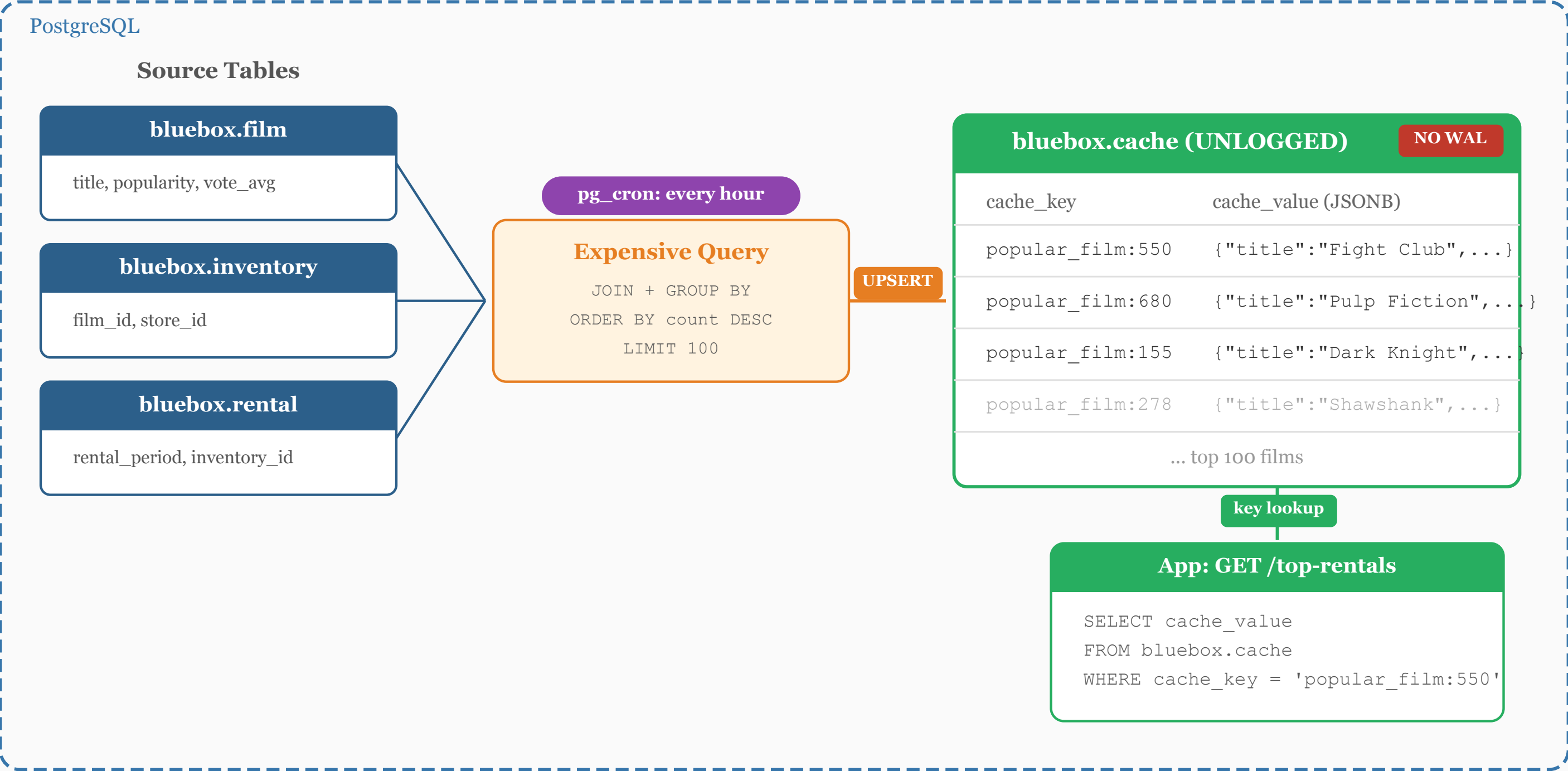
The Problem: Slow Film Catalog

- The "Top Rentals" page queries rentals + films + inventory every load
- Same 100 popular films served thousands of times per hour
- Data only needs to be fresh-ish — hourly is fine
- **Idea:** pre-compute the result, serve from a fast lookup table
- Don't want to add Redis just for this

Caching: The Usual Suspects

- **Redis / Valkey** — in-memory, sub-millisecond
- **Memcached** — simple KV cache
- **Application-level caches** — in-process

UNLOGGED Table Cache Pattern



Postgres Has: UNLOGGED Tables

```
-- An UNLOGGED table skips WAL (Write-Ahead Log)
-- = Faster writes, but data is LOST on crash
-- Sound familiar? That's exactly how Redis works!

-- Your "cache" IS a table. Key-value pairs.
CREATE UNLOGGED TABLE bluebox.cache (
    cache_key TEXT PRIMARY KEY,      -- the "key"
    cache_value JSONB NOT NULL,      -- the "value" (any shape)
    expires_at TIMESTAMPTZ NOT NULL -- TTL
        DEFAULT now() + interval '1 hour'
);

CREATE INDEX idx_cache_expires ON bluebox.cache (expires_at);
```

Regular Table vs Cache Table

bluebox.film (LOGGED, normal table)			
film_id	title	popularity	vote_avg
550	Fight Club	73.4	8.4
680	Pulp Fiction	88.1	8.5

↑ WAL-protected, crash-safe, replicated

bluebox.cache (UNLOGGED, cache table)		
cache_key	cache_value (JSONB)	expires_at
film:550	{"title":"Fight Club"}	2026-09-01
top_rentals:NY	[{"film_id":550},...]	2026-08-31

↑ NO WAL, fast writes, LOST on crash (like Redis)

Both are queryable with SQL. Both support indexes. The difference is durability.

Basic Cache Operations

```
-- SET (upsert)
INSERT INTO bluebox.cache (cache_key, cache_value, expires_at)
VALUES ('film:550', '{"title":"Fight Club","rating":8.4}',
        now() + interval '24 hours')
ON CONFLICT (cache_key) DO UPDATE
SET cache_value = EXCLUDED.cache_value,
    expires_at = EXCLUDED.expires_at;

-- GET
SELECT cache_value FROM bluebox.cache
WHERE cache_key = 'film:550'
    AND expires_at > now();

-- DELETE expired
DELETE FROM bluebox.cache WHERE expires_at < now();
```

Bluebox: Cache Popular Films

```
-- Warm the cache with top-rented films
INSERT INTO bluebox.cache (cache_key, cache_value)
SELECT
    'popular_film:' || f.film_id,
    jsonb_build_object(
        'title', f.title,
        'popularity', f.popularity,
        'rental_count', count(r.rental_id)
    )
FROM bluebox.film f
JOIN bluebox.inventory i USING (film_id)
JOIN bluebox.rental r USING (inventory_id)
GROUP BY f.film_id
ORDER BY count(r.rental_id) DESC
LIMIT 100
ON CONFLICT (cache_key) DO UPDATE
SET cache_value = EXCLUDED.cache_value;
```

Caching: Tradeoffs

Postgres **UNLOGGED**

Use Redis when...

Moderate read loads

Sub-ms latency critical

Already have Postgres

Millions of ops/sec

Same backup/monitoring

Built-in eviction policies

JSONB flexibility

Distributed cache needed

Transactional with app data

Pub/sub on cache keys

Document Store & APIs

JSONB

The Problem: Messy Film Credits

- TMDB API returns cast and crew as nested JSON arrays
- Each film has a different number of actors, directors, producers
- Need to search inside these arrays ("find all Tom Hanks films")
- Don't want a separate document store for one API's data

Document Store: The Usual Suspects

- **MongoDB** — document-first database
- **CouchDB** — JSON + HTTP API
- **DynamoDB** — AWS managed NoSQL

Why NoSQL Got Popular

1. **Schema flexibility** — no ALTER TABLE migrations
2. **Developer speed** — people loved JSON when it came out
3. **APIs** — obvs
4. **Postgres features were limited** - lots of support has been added over the last decade

Postgres Has: JSONB

```
-- Different "schemas" in the SAME table, SAME column  
-- This is the MongoDB appeal – and JSONB does it too
```

```
CREATE TABLE bluebox.api_events (  
    id BIGSERIAL PRIMARY KEY,  
    event_type TEXT,  
    payload JSONB  
);
```

```
INSERT INTO bluebox.api_events (event_type, payload) VALUES  
( 'signup', '{"user": "jane", "plan": "pro", "referral": "google"}' ),  
( 'purchase', '{"user": "jane", "items": [{"sku": "DVD-001", "qty": 2}],  
    "total": 9.99, "coupon": null}' ),  
( 'support', '{"user": "jane", "ticket_id": 4521,  
    "tags": ["billing", "urgent"],  
    "metadata": {"browser": "Chrome", "os": "macOS"}}' );
```

JSONB Operators

Operator	Returns	Example
->	JSON object	payload->'metadata'
->>	Text value	payload->>'user' → 'jane'
->0	Array element	"cast"->0 (first item)
@>	Containment match	"cast" @> '[{"name": "Tom Hanks"}]'
?	Key exists?	payload ? 'tags'
jsonb_array_elements()	Unnest array to rows	FROM jsonb_array_elements("crew")
jsonb_array_length()	Array size	jsonb_array_length("cast")

Chain them: payload->'items' ->0->>'sku'

Generalized Inverted Indexes (GIN) for JSONB

-- Index the entire JSONB document

```
CREATE INDEX idx_credits_cast_gin  
ON staging.film_credits USING GIN ("cast");
```

-- Now containment queries are fast!

-- This uses the index:

```
SELECT film_id FROM staging.film_credits  
WHERE "cast" @> ' [{"name": "Scarlett Johansson"}] ';
```

Bluebox: Flexible Film Metadata

```
-- Store API response data alongside relational  
-- Real pattern: keep structure for queries, JSONB for the rest
```

```
SELECT f.title, f.release_date,  
       fc."cast" ->> 'name' AS star,  
       jsonb_array_length(fc."cast") AS cast_size,  
       jsonb_array_length(fc."crew") AS crew_size  
FROM bluebox.film f  
JOIN staging.film_credits fc USING (film_id)  
WHERE f.popularity > 100  
ORDER BY f.popularity DESC  
LIMIT 10;
```

Search with regex jsonb_path

```
-- JSONPath: powerful querying inside documents
SELECT film_id,
       jsonb_path_query_array(
           "cast",
           '$[*] ? (@.name like_regex "^Tom")'
       ) AS toms_in_cast
FROM staging.film_credits
WHERE jsonb_path_exists(
       "cast",
       '$[*] ? (@.name like_regex "^Tom")'
)
LIMIT 5;
```

Blog: "I Replaced MongoDB with a Single Postgres Table"

"Postgres and its JSONB data type can replace MongoDB for most NoSQL use cases. Get schema flexibility, ACID compliance, and fast queries from a single table."

— *userjot.com/blog/postgres-jsonb-vs-mongodb*

Document Store: Tradeoffs

Postgres JSONB

Hybrid relational + document

ACID across documents

Joins between JSON & tables

One database to operate

GIN indexes are fast

Use MongoDB when...

Pure document workloads

Horizontal sharding from day 1

> 1TB of documents

Aggregation pipeline preferred

Write-heavy, high cardinality

Spatial Data

PostGIS

Points, Lines, Polygons



River represented
as a Line



Well locations represented
as points



Lake represented as
polygon

Geospatial Data in Postgres

```
-- Bluebox already has geography columns!
-- customer.geog and store.geog are Point(4326)

SELECT full_name,
       ST_AsText(geog) AS location
FROM bluebox.customer
LIMIT 3;
-- Jane Smith | POINT(-73.9857 40.7484)

-- Which store is closest to customer 42?
SELECT s.store_id, s.street_name,
       round(ST_Distance(c.geog, s.geog)::numeric) AS distance_meters
FROM bluebox.customer c
CROSS JOIN bluebox.store s
WHERE c.customer_id = 42
ORDER BY c.geog <-> s.geog -- KNN operator!
LIMIT 3;
```

WKB vs Traditional Geo Data

Open Source Geo Stack

Extensions

PostGIS — geometry, geography, spatial indexing

postgistigergeocoder — US address geocoding

pgrouting — shortest path, Dijkstra, A*

h3-pg — Uber's hexagonal spatial index

pgpointcloud — LiDAR point cloud storage

postgis_raster — raster data in the DB

Tools & Libraries

pg_tileserv — MVT vector tiles from PostGIS

pg_featureserv — GeoJSON via OGC API

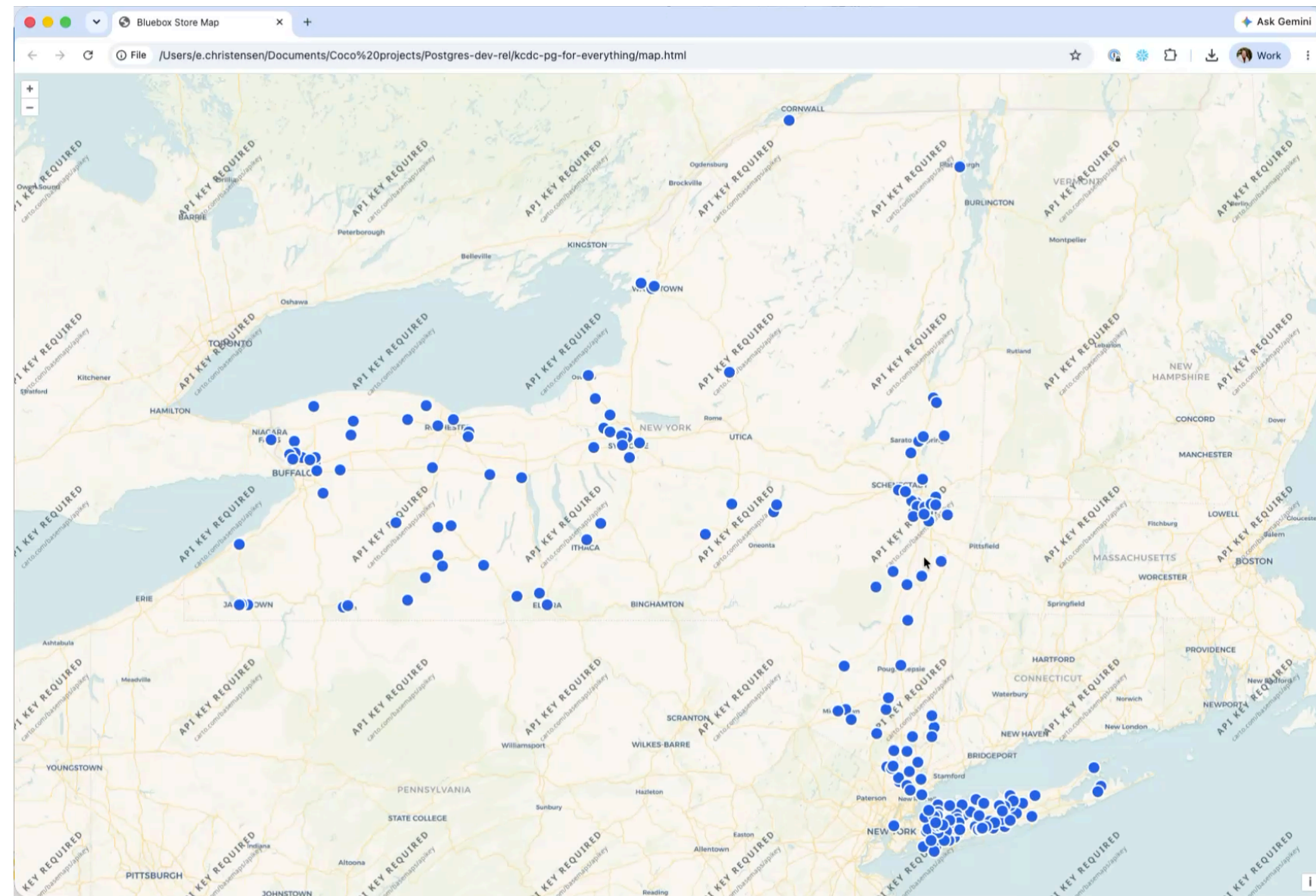
OpenLayers — JS map library (no API key)

QGIS — desktop GIS, native PostGIS connection

GDAL/OGR — format conversion, ETL

GeoServer — OGC WMS/WFS server

pg_featureserv → Bluebox Store Map



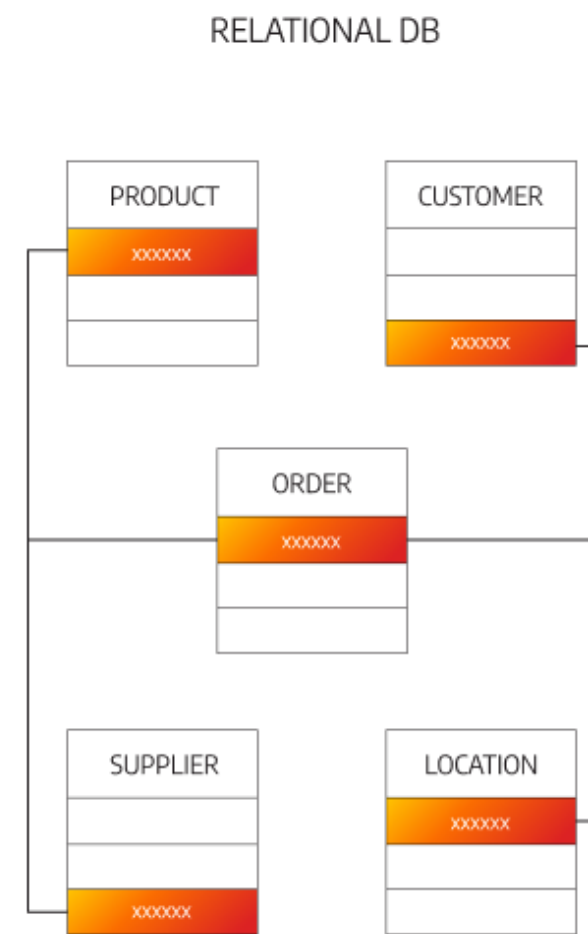
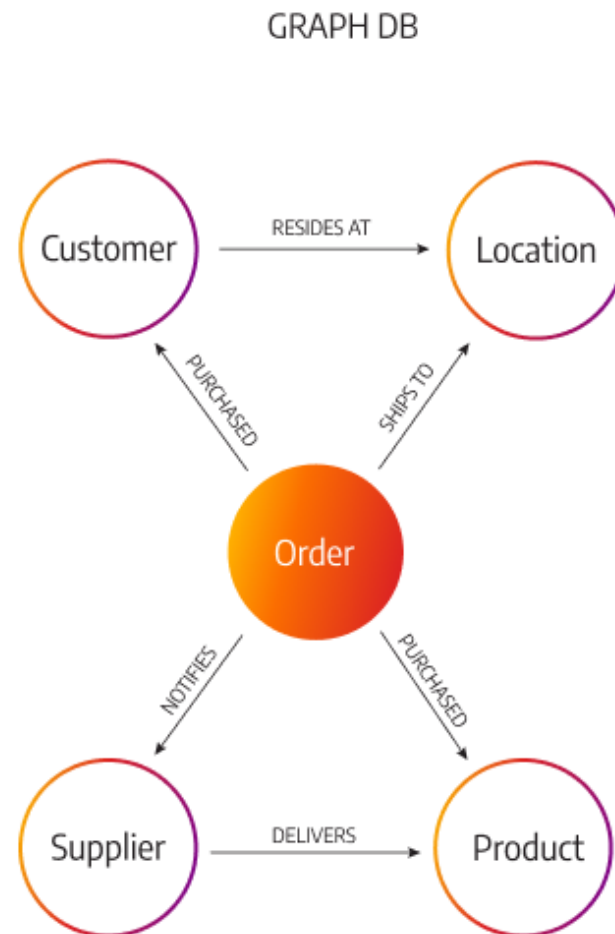
Graph Queries

Property Graph (pg19) & Apache AGE

Graph DBs

- Complex relationships: social networks, fraud detection, knowledge graphs, personalization
- Interconnected data with unknown number of hops between relationships (friend-of-friend)
- Rapid schema changes

Graph vs Relational



Property Graph vs Cypher

	Property Graph (PG19)	Cypher (Apache AGE)
Standard	ISO SQL:2023	openCypher (Neo4j-originated)
Storage	View over existing tables	Separate graph store (agtype)
Query style	<code>MATCH (a IS person)-[e IS cast]->(b IS film)</code>	<code>MATCH (a:Person)-[:ACTED_IN]->(b:Film)</code>
Variable-length paths	Not yet (PG19)	Yes — <code>[:ACTED_IN*1..4]</code>
Graph algorithms	No built-in	APOC, GDS (Neo4j)
Data loading	Zero — uses your tables	Must import into graph

The Problem: Actor Connections

- "Fans of Kevin Bacon also watched..." – recommend through cast overlap

Graph Queries: The Usual Suspects

- **Neo4j** — native graph, Cypher language
- **Amazon Neptune** — managed graph
- **PuppyGraph** — graph query layer over existing DBs (no ETL)

The Old Postgres Way: Recursive CTEs

```
-- Find actor collaboration chains
WITH RECURSIVE collab AS (
    -- Start: films with actor A
    SELECT fc2.person_id, 1 AS depth
    FROM bluebox.film_cast fc1
    JOIN bluebox.film_cast fc2 USING (film_id)
    WHERE fc1.person_id = 500 -- Tom Hanks
        AND fc2.person_id != 500
    UNION
    -- Recurse: films with those actors...
    SELECT fc2.person_id, c.depth + 1
    FROM collab c
    JOIN bluebox.film_cast fc1 ON fc1.person_id = c.person_id
    JOIN bluebox.film_cast fc2 USING (film_id)
    WHERE fc2.person_id != 500 AND c.depth < 3
)
SELECT DISTINCT person_id, min(depth) FROM collab GROUP BY 1;
```

NEW in PG19: SQL/PGQ

```
-- Define a graph over existing tables (it's a VIEW!)
CREATE PROPERTY GRAPH movie_graph
  VERTEX TABLES (
    bluebox.person KEY (person_id)
      PROPERTIES (person_id, name, popularity),
    bluebox.film KEY (film_id)
      PROPERTIES (film_id, title, release_date)
  )
  EDGE TABLES (
    bluebox.film_cast KEY (film_id, person_id)
      SOURCE KEY (person_id) REFERENCES person (person_id)
      DESTINATION KEY (film_id) REFERENCES film (film_id)
      PROPERTIES (film_character)
  );
```

SQL/PGQ: 1 Hop (co-stars)

```
SELECT * FROM GRAPH_TABLE (movie_graph
    MATCH (a IS person)-[IS film_cast]->(f IS film)
        <-[IS film_cast]-(b IS person)
    WHERE a.name = 'Kevin Bacon'
        AND b.person_id != a.person_id
    COLUMNS (b.name AS costar, f.title AS film)
) ORDER BY film LIMIT 5;
```

costar		film
Tom Hanks		Apollo 13
Andie MacDowell		Beauty Shop
John Carroll Lynch		Crazy, Stupid, Love.
Evan Rachel Wood		Digging to China
Kevin Denson		Footloose

SQL/PGQ: 2 Hops — Kevin Bacon to Sally Field

```
-- Must hardcode every node and edge in the chain
SELECT * FROM GRAPH_TABLE (movie_graph
    MATCH (a IS person)-[IS film_cast]->(f1 IS film)
        <-[IS film_cast]-(mid IS person)
        -[IS film_cast]->(f2 IS film)
        <-[IS film_cast]-(b IS person)
    WHERE a.name = 'Kevin Bacon'
        AND b.name = 'Sally Field'
        AND mid.person_id != a.person_id
        AND b.person_id != mid.person_id
        AND mid.popularity > 20
    COLUMNS (f1.title AS film_1, mid.name AS via_actor,
              f2.title AS film_2)
) LIMIT 5;
```

film_1	via_actor	film_2
Apollo 13	Tom Hanks	Forrest Gump
Crazy, Stupid, Love.	Emma Stone	The Amazing Spider-Man
Hollow Man	Elisabeth Shue	Soapdish

Apache AGE: Any Depth

```
-- Same question, any depth – change one number
SELECT * FROM cypher('movie_graph', $$
    MATCH p = (a:Person)-[:ACTED_IN*1..6]-(b:Person)
    WHERE a.name = 'Kevin Bacon'
    AND b.name = 'Sally Field'
    RETURN length(p) AS hops,
           [n IN nodes(p) | coalesce(n.name, n.title)] AS chain
    ORDER BY hops LIMIT 3
$$) AS (hops agtype, chain agtype);
```

hops | chain

```
-----+-----
4      | ["Kevin Bacon","Apollo 13","Tom Hanks","Forrest Gump","Sally Field"]
6      | ["Kevin Bacon","She's Having a Baby","Bill Murray",
      | "Ghostbusters: Afterlife","Olivia Wilde","Her","Sally Field"]
```

PG19 SQL/PGQ vs Apache AGE

- **SQL/PGQ**: graph is a view over your tables — zero ETL, zero sync
- **Apache AGE**: separate graph store — must load data, but gets variable-length paths
- For fixed-depth patterns (1-2 hops): **SQL/PGQ is simpler**
- For deep traversals or "find any path": **AGE (or Neo4j) is necessary**
- Both run inside Postgres — no separate database server

Blog: "PostgreSQL 19 Is a Graph Database Now"

"If you have ever spun up Neo4j because you had a vague intuition that your data was graph-shaped, this is the release that lets you reconsider."

— *thebuild.com* (Christophe Pettus)

Choosing a graph only db

- Is your entire data model a graph?
- Do you need complex algorithms and deep traversals?
- If you just have one graph thing, try Postgres as a start

AI & Vector Search

pgvector

The Problem: "Films Like This"

- Customer describes what they want: "a heist movie with a clever twist"
- Keyword search fails — need meaning, not exact words
- Recommend films similar to one they just watched
- Combine similarity with filters: "sci-fi from the 2020s"

Vector Search: The Usual Suspects

- **Pinecone** — managed vector DB
- **Weaviate** — open-source vector DB
- **LanceDB** - open-source serverless vector DB

Postgres Has: pgvector

```
CREATE EXTENSION vector;

-- Add embedding column to films
ALTER TABLE bluebox.film
ADD COLUMN embedding vector(1536); -- OpenAI dimensions

-- Create HNSW index for fast similarity
CREATE INDEX ON bluebox.film
USING hnsw (embedding vector_cosine_ops);
```

Similarity Search

```
-- Find films similar to a query embedding
SELECT title, overview,
       1 - (embedding <=> $1) AS similarity
FROM bluebox.film
WHERE embedding IS NOT NULL
ORDER BY embedding <=> $1 -- cosine distance
LIMIT 5;

-- $1 = embedding of "a heist movie with a clever twist"
-- Returns: Ocean's Eleven, The Italian Job, Inside Man...
```

The Killer Feature: Hybrid Queries

```
-- Vector search + relational filters in ONE query
-- "Find similar sci-fi films from the 2020s"
SELECT title, release_date,
       1 - (embedding <=> $1) AS similarity
FROM bluebox.film
WHERE release_date >= '2020-01-01'
      AND 878 = ANY(genre_ids) -- sci-fi genre
ORDER BY embedding <=> $1
LIMIT 10;
```

Blog: "Why We Replaced Pinecone with pgvector"

"After weeks of experimentation, we made the decision to replace Pinecone entirely with pgvector. Since HNSW was introduced it now outperforms all three pod types."

— *Confident AI (confident-ai.com)*

Vector Search: Tradeoffs

pgvector

Use a dedicated vector DB when...

< 50M vectors

> 100M vectors

Vectors + relational data

Pure vector workloads

High concurrency / QPS

Single-request latency critical

One INSERT = atomic

Write-heavy embedding pipelines

Free, open-source

Need managed auto-scaling

Existing PG tooling works

* Built-in vectorization APIs

Full-Text Search

tsvector, pg_trgm, ParadeDB

The Problem: Film Search Box

- Customer types "space adventure" — rank results by relevance
- Customer types "Inception" — still find "Inception" (typo tolerance)
- "running" should match "run" and "ran" (stemming)
- Title matches should rank higher than plot description matches

Full-Text Search: The Usual Suspects

- **Elasticsearch** — distributed search & analytics
- **Algolia** — managed search API
- **Meilisearch** — fast, typo-tolerant
- **Typesense** — lightweight alternative

Postgres Has: Built-in FTS

Extension	What it does	Ships with PG?
tsvector/tsquery	Stemming, ranking, phrase search	Core (always available)
pg_trgm	Trigram similarity, fuzzy match, type-ahead	Contrib (ships with PG)
fuzzystrmatch	Soundex, Daitch-Mokotoff, Levenshtein distance	Contrib (ships with PG)
unaccent	Strip diacritics (café → cafe)	Contrib (ships with PG)

Postgres Built-in FTS

```
-- Bluebox films already have a tsvector column!  
-- It's a GENERATED column:  
-- fulltext tsvector GENERATED ALWAYS AS (  
--   to_tsvector('english', title || ' ' || overview)  
-- ) STORED
```

```
SELECT title, ts_rank(fulltext, q) AS rank  
FROM bluebox.film, to_tsquery('english', 'space & adventure') q  
WHERE fulltext @@ q  
ORDER BY rank DESC  
LIMIT 5;
```

How tsvector Works

-- Text → normalized tokens with positions

```
SELECT to_tsvector('english',  
    'The PostgreSQL database is running on AWS');
```

-- Result: 'aws':6 'databas':3 'postgresql':2 'reliabl':8 'run':5

-- (stems + positions; stop words removed)

-- Query → normalized search expression

```
SELECT to_tsquery('english', 'databases & running');
```

-- Result: 'databas' & 'run'

-- Match! Both stems present in the tsvector.

Weighted Search (title > overview)

```
-- Weight title higher than overview
SELECT title,
       ts_rank(
           setweight(to_tsvector('english', title), 'A') ||
           setweight(to_tsvector('english', overview), 'B'),
           query
       ) AS rank
FROM bluebox.film,
     websearch_to_tsquery('english', 'dark knight returns') AS query
WHERE (
    setweight(to_tsvector('english', title), 'A') ||
    setweight(to_tsvector('english', overview), 'B')
) @@ query
ORDER BY rank DESC LIMIT 5;
```

Typo Tolerance with pg_trgm

```
-- pg_trgm: trigram similarity for fuzzy matching
CREATE INDEX idx_film_title_trgm
ON bluebox.film USING GIN (title gin_trgm_ops);

-- "Did you mean?" when FTS returns nothing
SELECT title, similarity(title, 'Inception') AS sim
FROM bluebox.film
WHERE title % 'Inception' -- trigram similarity > 0.3
ORDER BY sim DESC
LIMIT 5;
-- Returns: "Inception" (similarity: 0.77)
```

Autocomplete / Type-ahead

```
-- Fast prefix search with trigrams
SELECT title
FROM bluebox.film
WHERE title ILIKE 'the god%'
ORDER BY popularity DESC
LIMIT 5;
-- "The Godfather", "The Godfather Part II", ...

-- With GIN trigram index, ILIKE is indexed!
```

Real-World: Search-as-you-type for 54M Names

Caktus Group built last-name type-ahead search for an entire US state — 54 million names — using only pg_trgm, Soundex, and Daitch-Mokotoff inside Django.

— *DjangoCon US 2026*

- "Smith" matches Smyth, Smythe, Smidt via Soundex
- Daitch-Mokotoff handles Slavic/European names (Weiss ↔ Weiß)

Full-Text Search: Tradeoffs

Postgres FTS

< 10M documents

Search is a feature, not the product

Use Elasticsearch when...

Hundreds of millions of docs

Search IS the product

OLAP & Analytics

pg_duckdb & Window Functions

The Problem: Revenue Reporting

- Bluebox management wants month-over-month revenue growth
- Rank top customers per store for loyalty rewards
- Revenue percentiles — which films earn the most?
- Run these reports on the same DB serving the app

OLAP: The Usual Suspects

- **ClickHouse** — columnar OLAP, petabyte scale
- **DuckDB** — embedded OLAP, "SQLite for analytics"
- **BigQuery** — serverless analytics

Postgres Has: Powerful Analytics SQL

```
-- Running total of revenue by month
SELECT
    date_trunc('month', payment_date) AS month,
    sum(amount) AS monthly_revenue,
    sum(sum(amount)) OVER (ORDER BY date_trunc('month', payment_date))
        AS running_total
FROM bluebox.payment
GROUP BY month
ORDER BY month;
```

month	monthly_revenue	running_total
2024-01	405977.91	405977.91
2024-02	385904.78	791882.69
2024-03	394640.88	1186523.57
2024-04	29477.87	1216001.44
2024-11	217612.47	1433613.91
2024-12	450786.74	1884400.65

Window Functions: Rental Patterns

```
-- Rank customers by rental frequency per store
SELECT
  c.full_name,
  s.street_name AS store,
  count(*) AS rentals,
  rank() OVER (
    PARTITION BY r.store_id
    ORDER BY count(*) DESC
  ) AS store_rank
FROM bluebox.rental r
JOIN bluebox.customer c USING (customer_id)
JOIN bluebox.store s ON r.store_id = s.store_id
GROUP BY c.full_name, s.street_name, r.store_id
HAVING count(*) > 5
ORDER BY store_rank
LIMIT 20;
```

Revenue Analytics

```
-- Month-over-month growth rate
WITH monthly AS (
    SELECT
        date_trunc('month', payment_date) AS month,
        sum(amount) AS revenue
    FROM bluebox.payment
    GROUP BY 1
)
SELECT month,
    revenue,
    lag(revenue) OVER (ORDER BY month) AS prev_month,
    round(
        (revenue - lag(revenue) OVER (ORDER BY month))
        / lag(revenue) OVER (ORDER BY month) * 100, 1
    ) AS growth_pct
FROM monthly
ORDER BY month;
```

Apache DataSketches

Sketches are probabilistic data structures — approximate answers in constant time and constant space.

- CPC / HLL - Distinct counting (how many unique customers?)
- Theta - Distinct counting with set operations (intersection, difference)
- KLL - Quantiles/median
- Frequent Strings - Top-N / heavy-hitter detection

Apache DataSketches

- ~1-4 KB per sketch regardless of input size
- Error < 2%, **mergeable** — union daily sketches into weekly without re-scanning

DataSketches: Pre-Aggregate Then Merge

```
CREATE EXTENSION datasketches;

-- 1. Build sketches per store per day during ETL
CREATE TABLE bluebox.rental_sketches AS
SELECT store_id,
       date_trunc('day', lower(rental_period)) AS day,
       count(*) AS rentals,
       cpc_sketch_build(customer_id) AS unique_customers_sketch
FROM bluebox.rental r
JOIN bluebox.inventory i USING (inventory_id)
GROUP BY store_id, day;

-- 2. Query: merge sketches (instant, regardless of raw data size)
SELECT store_id,
       sum(rentals) AS total_rentals,
       cpc_sketch_get_estimate(
         cpc_sketch_union(unique_customers_sketch)
       ) AS unique_customers
FROM bluebox.rental_sketches
WHERE day BETWEEN '2026-01-01' AND '2026-01-31'
GROUP BY store_id;
```

Blog: Approximate Answers in PostgreSQL

"When your table has 500 million rows and someone asks 'how many unique users last month?' — you don't need an exact answer. You need an answer in 50ms."

snowflake.com/en/blog/engineering/postgres-count-distinct-approximation

OLAP: Tradeoffs

Postgres Analytics

**Use ClickHouse/DuckDB
when...**

Mixed OLTP + analytics

Pure analytical workloads

Rollups will help

Columnar compression needed

pg_lake

Postgres as a Lakehouse

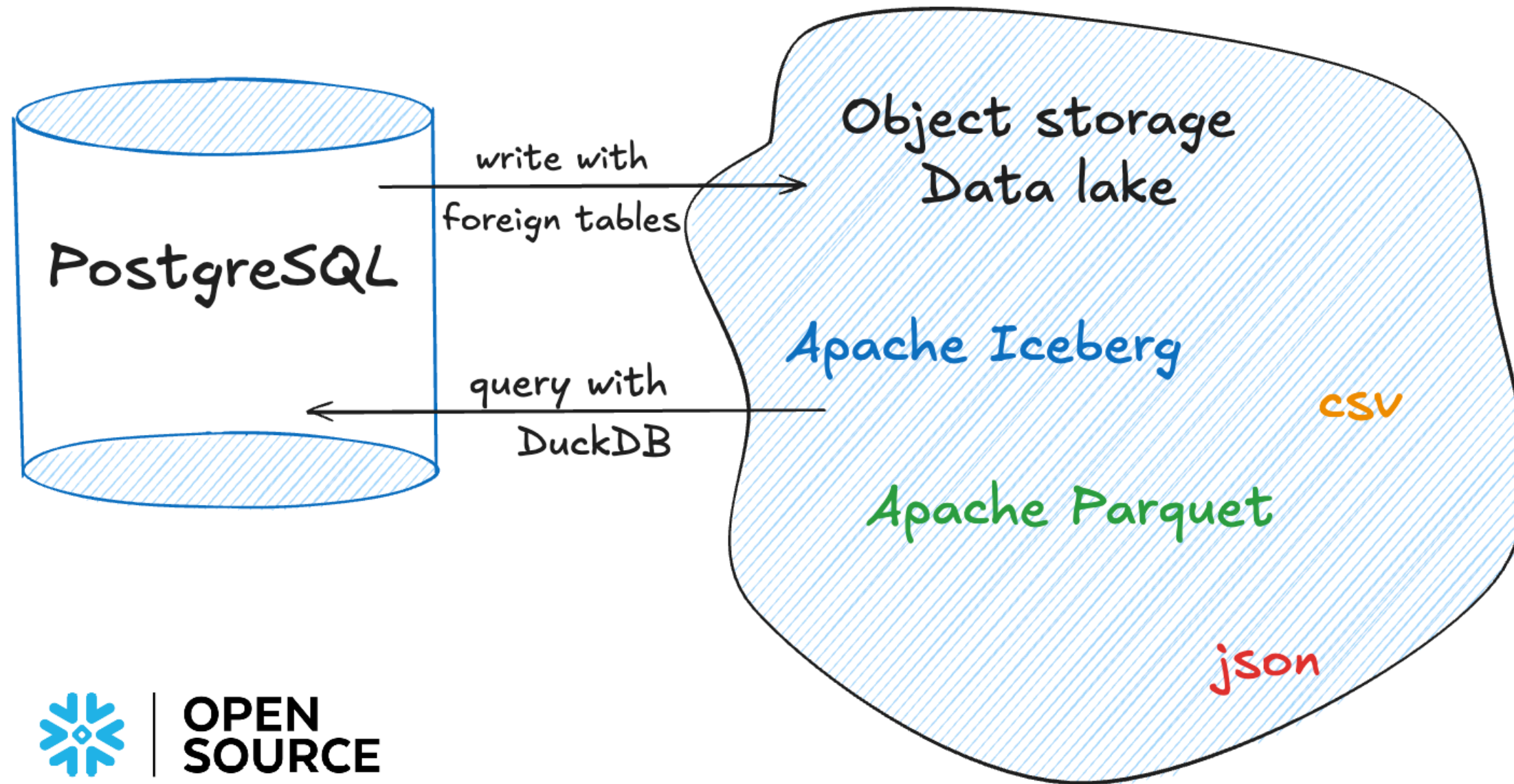
The Problem: External Data Integration

- Marketing team has Amazon review data in S3 as Parquet
- Analytics team runs Spark jobs on the data lake
- You need to join external data with Bluebox rental data
- Don't want a separate ETL pipeline to load S3 data into Postgres
- Want to share Postgres data back out in an open format

Data Lakehouse: The Usual Suspects

- **DuckDB** - query object stores
- **Spark + Iceberg** — distributed processing on data lakes
- **Snowflake** — managed warehouse with Iceberg support
- **Databricks** — unified analytics on Delta Lake

pg_lake: Postgres for lakehouse



pg_lake: What Is It?

Postgres for Iceberg and Data Lakes

- Create and modify **Iceberg tables** directly from PostgreSQL
- Query **Parquet, CSV, JSON** files in object storage (S3, GCS)
- Export query results back to object storage
- Combine **heap tables + Iceberg + external files** in the same SQL
- Uses **DuckDB's query engine** under the hood for fast execution

pg_lake: Getting Started

```
-- Install the extension
CREATE EXTENSION pg_lake CASCADE;
-- Installs: pg_lake_table, pg_lake_engine,
--           pg_lake_iceberg, pg_lake_copy

-- Create an Iceberg table from existing data
CREATE TABLE analytics.film_metrics
USING iceberg AS
SELECT film_id, title, popularity, vote_average
FROM bluebox.film
WHERE release_date >= '2020-01-01';
```

pg_lake: Query External Files

-- Read a Parquet file from S3 directly

```
SELECT * FROM read_parquet('s3://my-bucket/exports/rentals.parquet')  
LIMIT 10;
```

-- Read CSV with automatic schema inference

```
SELECT * FROM read_csv('s3://my-bucket/data/customers.csv')  
WHERE state_id = 'NY';
```

-- Join external data with local tables

```
SELECT f.title, p.total_revenue  
FROM bluebox.film f  
JOIN read_parquet('s3://analytics/revenue.parquet') p  
  ON f.film_id = p.film_id;
```

pg_lake: Join S3 Data with Postgres Tables

```
-- Foreign table pointing at public Amazon reviews on S3
CREATE FOREIGN TABLE amazon_video_reviews ()
SERVER pg_lake
OPTIONS (path
    's3://amazon-reviews-pds/parquet/product_category=Digital_Video_Download/');

-- Films with great Amazon reviews but low Bluebox rentals
SELECT f.title,
       round(avg(r.star_rating), 1) AS avg_stars,
       count(r.*) AS review_count,
       coalesce(rentals.cnt, 0) AS bluebox_rentals
FROM amazon_video_reviews r
JOIN bluebox.film f ON f.title = r.product_title
LEFT JOIN (
    SELECT i.film_id, count(*) AS cnt
    FROM bluebox.rental ren
    JOIN bluebox.inventory i USING (inventory_id)
    GROUP BY i.film_id
) rentals ON rentals.film_id = f.film_id
GROUP BY f.title, rentals.cnt
HAVING avg(r.star_rating) >= 4
ORDER BY bluebox_rentals ASC LIMIT 10;
```

pg_lake: Why It Matters

- **One SQL interface** for operational + analytical data
- No ETL pipeline from Postgres to data lake
- Iceberg tables readable by **Spark, Trino, Snowflake**
- DuckDB engine handles columnar scans efficiently
- PostGIS spatial data in Iceberg, shared with analytics team

The "Postgres for Everything" philosophy extended to the data lake.

pg_lake + PostGIS

```
-- Spatial data in a lakehouse!
-- pg_lake supports GDAL formats: GeoJSON, Shapefiles

-- Read GeoJSON from S3
SELECT * FROM read_gdal('s3://geo-data/boundaries.geojson');

-- Create an Iceberg table with spatial data
CREATE TABLE geo.store_coverage
USING iceberg AS
SELECT store_id, street_name,
       ST_Buffer(geog::geometry, 5000) AS coverage_area
FROM bluebox.store;
```

Summary

When to stay, when to leave

The Decision Framework

Stay with Postgres when:

1. Your scale is moderate (< millions of events/sec)
2. You value operational simplicity
3. Your data has relational connections
4. You need ACID guarantees
5. One less service = one less failure mode

The Decision Framework

Leave Postgres when:

1. You're hitting **millions of ops per second**
2. You need **distributed, multi-region** by default
3. The specialized tool has **10x better performance** for your exact workload
4. Your team **already has expertise** in the specialized tool

The "Postgres Is Enough" Philosophy

"For 99% of the use cases Postgres is enough and the best choice in my opinion."

— *r/ExperiencedDevs* (400+ upvotes)

But Also...

"If you want to make it to the front page of HackerNews, arguing that 'Postgres is enough' may get you there; but if you actually want to solve your real-world problems, use the right tool for the job."

— *Gunnar Morling (Debezium creator)*

The Real Answer

Scale	Recommendation
Startup / side project	Just use Postgres
Growing company	Postgres + maybe 1 specialized tool
Enterprise / hyperscale	Right tool for each job

The question isn't "can Postgres do it?" — it's "at what scale does it stop being enough?"

Please let me know
what you think of
this talk!



Tools We Covered

Feature	Extension/Method
Job queues	SKIP LOCKED
Pub/Sub	LISTEN/NOTIFY
Webhooks	pgsql-http (Paul Ramsey)
Caching	UNLOGGED tables
Documents	JSONB + GIN
Spatial	PostGIS + pg_tileserv
Graphs	SQL/PGQ (pg19) + Apache AGE
Vectors	pgvector
Full-text search	tsvector + pg_trgm + ParadeDB
OLAP	Window functions + Apache DataSketches
Lakehouse	pg_lake (Iceberg + DuckDB)

Thank You!

Elizabeth Christensen

@sqlliz

Questions? Let's talk Postgres!

